

CIS 761 — Database Management Systems

Exam 2 — Worked Solutions & Concept Guide

This document solves every question on Exam 2 and explains the underlying database concepts in detail. Section A is a concept primer that covers *everything* the exam touches: triggers, stored procedures, user-defined functions, transactions and the ACID properties, isolation levels and concurrency anomalies, indexes, B+ trees (including step-by-step insertion and deletion rules), and the MapReduce programming model. Sections I–IV then present each exam question with the correct answer in green and a thorough explanation in plain English.

Section	Topic	Points
A	Concept Primer (background reading)	—
I	True / False Questions	10
II	Multiple-Choice Questions	10
III	Triggers & MapReduce	15
IV	Indexes and B+ Trees	15
	Total exam points	50

Section A — Concept Primer

Read this section first if any of the topics feel unfamiliar. Each subsection introduces the concept in plain language, then states the technical rules you need to apply on the exam.

A.1 — Triggers

A **trigger** is a named block of SQL/PL-SQL code that the DBMS fires *automatically* in response to an INSERT, UPDATE, or DELETE event on a specific table or view. The user never calls a trigger directly; the database engine invokes it as part of the modifying statement.

Anatomy of a trigger

- **Event** — INSERT, UPDATE, or DELETE (and optionally the specific column for UPDATE OF col).
- **Timing** — BEFORE (run before the row change is applied; useful for validation/transformation) or AFTER (run after the change; useful for auditing/cascading actions).
- **Granularity** — FOR EACH ROW (row-level: fires once per affected row, exposes OLD and NEW pseudo-records) or FOR EACH STATEMENT (statement-level: fires once regardless of how many rows are touched).
- **Body** — a PL/SQL block (BEGIN ... END) containing any legal SQL plus control-flow (IF/LOOP/etc.).

Why triggers exist

- Enforce *complex* integrity constraints that go beyond CHECK / FOREIGN KEY (e.g. cross-table rules).
- Maintain derived data automatically (totals, counts, history tables).
- Audit logging — record who changed what and when.
- Cascading business actions (send notification, write into a queue, delete dependent records).

A.2 — Stored Procedures

A **stored procedure** is a named, pre-compiled block of SQL stored in the database. Unlike triggers, stored procedures are called *explicitly* by an application (CALL proc(args) / EXEC proc args). They accept input and output parameters and may return result sets.

Benefits

- **Performance** — parsed and optimized once; the server caches the execution plan.
- **Data quality** — every application calls the *same* validated SQL, removing ad-hoc query bugs.
- **Security** — users can be granted EXECUTE on the procedure without being granted direct DML on the underlying tables. Procedures may run with definer privileges, hiding the schema.
- **Maintainability** — business logic lives in one place rather than copy-pasted across applications.

Most DBMS products separate the privilege to *create* procedures (CREATE PROCEDURE) from the privilege to *execute* them. A DBA grants CREATE PROCEDURE only to trusted developers; this prevents arbitrary users from injecting code into the schema.

A.3 — User-Defined Functions (UDFs)

A UDF is a named function stored in the database that can be called from inside SQL expressions. There are two kinds:

- **Scalar UDF** — returns *exactly one* scalar value (a number, string, date, etc.). Usable anywhere a scalar expression is allowed.
- **Table-valued UDF** — returns a result set; usable in the FROM clause like a view.

If you need to return multiple values, use either a table-valued function or a stored procedure with OUT parameters. A scalar UDF cannot return multiple scalar values — that is a frequent exam trap.

A.4 — Transactions and the ACID Properties

A **transaction** is a logical unit of work that takes the database from one consistent state to another. The DBMS guarantees the four ACID properties:

Property	Guarantee	Example
Atomicity	All operations succeed or none of them do.	Bank transfer either debits and credits, or leaves both rows alone.
Consistency	Each transaction takes the DB from a valid state to another valid state, up to before and after integrity constraint.	State is not another valid state before and after integrity constraint.
Isolation	Concurrent transactions appear to execute serially.	Serially, concurrent transfers don't see each other's intermediate writes.
Durability	Once committed, changes survive crashes.	After COMMIT, a power failure does not lose the update.

If you do not start an explicit transaction with `BEGIN TRANSACTION`, most DBMSes run in **auto-commit** mode: every individual SQL statement is wrapped in its own implicit `BEGIN...COMMIT` pair, so each statement is its own atomic unit.

A.5 — Isolation Levels and Concurrency Anomalies

The SQL standard defines four isolation levels. Stricter levels prevent more anomalies but reduce concurrency.

Anomaly	What goes wrong
Dirty read	T1 reads a value that T2 has written but not yet committed. If T2 rolls back, T1 saw a value that never existed.
Lost update	T1 and T2 both read $x=v$. Each computes a new value from v and writes back. The second write overwrites the first.
Non-repeatable read	T1 reads row R , T2 updates R and commits, T1 reads R again and gets a different value within the same transaction.
Phantom read	T1 runs a range query, T2 inserts a new row that satisfies the predicate and commits, T1 re-runs the same query and gets more rows.

Isolation level	Dirty read	Non-repeatable	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	prevented	possible	possible
REPEATABLE READ	prevented	prevented	possible
SERIALIZABLE	prevented	prevented	prevented

SERIALIZABLE is the strictest level and produces an execution equivalent to running the transactions one at a time. It prevents all anomalies — including lost updates and phantoms — but it acquires the most locks (or relies on heavy-weight snapshot validation), which can severely hurt throughput.

A.6 — Indexes

An **index** is an auxiliary data structure that maps key values to the locations of matching rows in a table. The DBMS uses an index to find rows without scanning the entire table, so queries that filter or join on indexed columns run much faster. The trade-off is that every `INSERT/UPDATE/DELETE` must also update the index, so writes get slower.

- **Clustered index** — the table is physically stored in index key order. Each table can have at most one. (In SQL Server it is the default for the primary key; in MySQL InnoDB the primary key always clusters.)
- **Non-clustered (secondary) index** — a separate structure whose leaves contain the key plus a pointer to the actual row. A table can have many.
- **Unique index** — automatically created by most DBMSes whenever you declare a UNIQUE or PRIMARY KEY constraint, because the index is what lets the engine check duplicates in $O(\log n)$ time.
- **B+ tree index** — supports equality *and* range queries. **Hash index** — supports equality only.

A.7 — B+ Trees

A B+ tree is the workhorse data structure for disk-resident indexes. It is a balanced multi-way search tree with these key properties:

- **All data lives in the leaves.** Internal nodes store only routing keys (separators).
- **Perfectly height-balanced.** Every root-to-leaf path has *exactly* the same length. Search, insert, and delete all run in $O(\log_d n)$ I/Os.
- **Leaves are linked.** Each leaf points to the next (usually doubly-linked), so a range scan is one downward traversal followed by a leaf-list walk.
- **Fan-out parameter d .** An internal node (other than the root) holds between d and $2d$ keys, i.e. between $d+1$ and $2d+1$ child pointers. A leaf holds between d and $2d$ data entries.

Insertion algorithm

- Walk from the root to the appropriate leaf.
- Insert the new key in the leaf in sorted order.
- If the leaf now has more than $2d$ keys, **split** it in half. *Copy up* the smallest key of the right half into the parent (the key remains in the right leaf as well — leaves keep all data).
- If an internal node overflows, split and **push up** the middle key into the parent (this key leaves the child entirely — internal nodes are not data).
- If the root splits, a brand new root is created above it. This is the only way the tree grows in height.

Deletion algorithm

- Walk to the leaf and remove the key.
- If the leaf still has $\geq d$ keys, you are done.
- Otherwise the leaf **underflows**. Try **redistribution**: borrow a key from a sibling that has $> d$ keys and update the separator key in the parent.
- If neither sibling can lend, **merge** with a sibling and pull the separator key down from the parent. The parent may now underflow; recurse upward.
- If merging propagates to the root and the root ends up empty, delete the root — the tree shrinks by one level.

A.8 — MapReduce

MapReduce is a programming model for processing very large data sets on a cluster. The user writes only two functions, map and reduce; the framework handles scheduling, parallelism, data movement, and fault tolerance.

Pipeline phases (in execution order)

- **Map** — each mapper reads input records as (k, v) and emits zero or more intermediate (k', v') pairs.

- **Combiner** (optional) — a mini-reducer that runs on the mapper's machine to pre-aggregate values for the same key, reducing network traffic. Must be associative and commutative.
- **Partitioner** — assigns each intermediate key to exactly one reducer (default = $\text{hash}(k') \bmod R$).
- **Shuffle & Sort** — the framework transfers intermediate data across the network and sorts it so that each reducer receives *all* values for its keys, grouped together and sorted by key.
- **Reduce** — for each intermediate key, the reducer receives $(k', \text{list-of-}v')$ and emits the final output.

Data locality is a core MapReduce design principle: the framework schedules mapper tasks on (or near) the cluster nodes that physically hold the input data blocks (in HDFS), so that the expensive part — moving bytes over the network — happens only during shuffle and not during map.

A mapper may emit **any number** of intermediate pairs per input record, including zero. A filter mapper, for example, emits nothing for records that fail its predicate.

Section I — True / False Questions

Q1. A trigger can only be used for audit purposes, i.e., to update a log file/table when the audited data file/table changes.

Answer: FALSE

Explanation. Triggers are far more versatile than auditing. Because the trigger body is an arbitrary PL/SQL block, it can enforce complex business rules, maintain referential integrity, cascade changes to related tables, send notifications, compute derived values, or block invalid operations. Auditing is just one common use case.

Q2. Quality of data can be improved by using stored procedures. Each user interaction will use the same set of SQL statements for transactions, ensuring consistency and accuracy.

Answer: TRUE

Explanation. Centralising business logic inside stored procedures means every application calls the *same* tested, validated code. Ad-hoc queries cannot bypass the validation rules, which improves consistency and accuracy of the stored data.

Q3. Not all users may be permitted to write stored procedures. Selected users are granted permission to write procedures. This ensures data security.

Answer: TRUE

Explanation. In every major DBMS (SQL Server, PostgreSQL, Oracle, MySQL) the CREATE PROCEDURE privilege is a separate grant from the DML privileges. A DBA gives CREATE PROCEDURE only to trusted developers, which limits the risk of malicious or buggy code entering the schema.

Q4. A user-defined function can return multiple scalar values.

Answer: FALSE

Explanation. A *scalar* UDF returns exactly one scalar value — that's what 'scalar' means. To return many values, use a table-valued function (which returns a result set) or a stored procedure with OUT parameters.

Q5. In a B+ tree the lengths of the paths from the root to all leaf nodes differ from each other by at most 1.

Answer: FALSE

Explanation. B+ trees are *perfectly* height-balanced: every root-to-leaf path has *exactly* the same length, not 'at most 1'. (The 'at most 1' rule describes AVL trees.) This zero-difference balance is what gives B+ trees worst-case $O(\log_d n)$ search time.

Q6. The DBMS automatically creates a non-clustered index on every unique attribute (or group of attributes) to enforce the UNIQUE KEY constraint.

Answer: TRUE

Explanation. When you declare UNIQUE, the engine has to detect duplicates on every insert/update. The cheapest way to do that is to maintain a unique index, so SQL Server, MySQL InnoDB, PostgreSQL, etc. all create one automatically. The index is non-clustered unless the unique column happens to be the primary key.

Q7. If you don't group statements into explicit transactions, SQL automatically treats each SQL statement as a separate transaction.

Answer: TRUE

Explanation. This is **auto-commit mode**: each statement is wrapped in an implicit BEGIN...COMMIT pair. Changes become permanent immediately after the statement, so you cannot roll back across multiple statements unless you opened an explicit transaction with BEGIN TRANSACTION.

Q8. Concurrency is a problem when data is being modified but not when two or more transactions simply read the same data.

Answer: TRUE

Explanation. Read-only transactions cannot conflict with one another because reading does not change state. All concurrency anomalies (dirty read, lost update, non-repeatable read, phantom) require at least one writer.

Q9. In MapReduce, a mapper must generate at least one key/value intermediate pair for each input key/value pair.

Answer: FALSE

Explanation. A mapper may emit **zero, one, or many** intermediate pairs per input record. A filter mapper, for instance, emits nothing when the input record fails its predicate.

Q10. To determine the frequency of each non-stopword (instead of every word), you need to change **ONLY** the map() function in the WordCount pseudocode.

Answer: TRUE

Explanation. Stopword filtering belongs entirely to the mapper: before emitting (word, 1), the mapper checks whether the word is in the stopwords list and skips it. The reducer, which simply sums the 1s for each key it receives, does not need any modification.

Section II — Multiple-Choice Questions

Q1. A B+ tree with four levels (including the root) — what is the maximum number of NEWLY CREATED nodes when inserting one key?

- a. 5
- b. 4
- c. 3
- d. 2

Answer: (b)

Explanation. The worst case is when a leaf split cascades all the way to the root. With 4 levels, splitting can happen at every level, and *each split creates exactly one new node*: one new sibling leaf, one new sibling at each internal level, and one new root when the original root splits. That is 4 new nodes total, one per level. Option (a) 5 would require 5 levels of splits.

Q2. A named set of SQL statements configured to run automatically when data modifications occur is called ____.

- a. aggregate function
- b. stored procedure
- c. trigger
- d. view

Answer: (c)

Explanation. A **trigger** fires automatically in response to INSERT/UPDATE/DELETE — no user calls it. A stored procedure runs only when explicitly invoked. An aggregate function (SUM, COUNT, ...) is a calculation. A view is a stored query.

Q3. If you delete and then recreate a stored procedure, function, or trigger...

- a. you delete the tables on which the object is based
- b. you disable access to the tables on which the object is based
- c. you delete the security permissions assigned to the deleted object
- d. none of the above

Answer: (c)

Explanation. Dropping any database object also drops the GRANT/REVOKE permissions attached to it. The underlying tables are not affected at all. After recreating the object, the DBA must re-grant the necessary EXECUTE permissions.

Q4. The purpose of an index in SQL is:

- a. to enforce data integrity constraints
- b. to store intermediate query results
- c. to improve the speed of data retrieval operations
- d. to perform calculations on aggregate data

Answer: (c)

Explanation. An index is a separate data structure (B+ tree, hash table, ...) that maps key values to row locations, letting the optimiser find matching rows without scanning the whole table. Integrity is enforced by constraints; intermediate results live in temp tables/CTEs; aggregations are done by aggregate functions.

Q5. A **lost update** occurs when:

- a. an insert by another transaction affects rows being updated
- b. a transaction selects uncommitted data
- c. two transactions read the same row and then update it based on the original values
- d. two SELECTs get different values because of an intervening update

Answer: (c)

Explanation. Classic lost-update: T1 reads $x = 100$, T2 reads $x = 100$, T1 writes $x = 150$, T2 writes $x = 120$. T1's write is overwritten — 'lost' — because T2 computed its new value from a stale read. Option (b) is a dirty read; option (d) is a non-repeatable read.

Q6. One drawback of using the SERIALIZABLE isolation level is:

- a. it can result in lost updates
- b. it allows too many transactions simultaneously
- c. it can cause severe performance problems
- d. it can cause security problems

Answer: (c)

Explanation. SERIALIZABLE is the strictest level. It either acquires range locks (2-phase locking) or relies on heavy-weight snapshot validation (SSI). Either way, concurrency drops sharply and throughput suffers. SERIALIZABLE actually *prevents* lost updates and has nothing to do with security.

Q7. The transaction $\text{read}(x)$; $x:=x-50$; $\text{write}(x)$; $\text{read}(y)$; $y:=y+50$; $\text{write}(y)$ — the constraint that $x+y$ stays constant is which ACID property?

- a. Atomicity
- b. Consistency
- c. Isolation
- d. Durability

Answer: (b)

Explanation. **Consistency** says the transaction must take the database from one valid state to another, preserving all integrity constraints. The invariant $x + y = \text{constant}$ is exactly such a constraint. Atomicity = all-or-nothing; Isolation = no interference between concurrent transactions; Durability = committed changes survive crashes.

Q8. Select the correct statement about MapReduce:

- **a.** MapReduce tries to place the data and the compute as close as possible
- **b.** Map Task is performed using Mapper() function
- **c.** Reduce Task is performed using Map() function
- **d.** All of the mentioned

Answer: (a)

Explanation. Data locality is a core design principle of Hadoop / MapReduce: mappers are scheduled on (or near) the nodes that already hold the input HDFS blocks, so input bytes do not have to travel across the network. Option (b) is loosely worded but the function is map(), not Mapper(); option (c) is just wrong (reduce tasks call reduce()); so (d) is wrong as well.

Q9. Which component maps input key/value pairs to intermediate key/value pairs?

- **a.** Mapper
- **b.** Reducer
- **c.** Combiner
- **d.** Partitioner

Answer: (a)

Explanation. By definition, the Mapper takes each input (k, v) and emits zero or more intermediate (k', v') pairs. The Reducer aggregates values that share an intermediate key; the Combiner is an optional local mini-reducer; the Partitioner just routes intermediate keys to the right reducer.

Q10. Put the MapReduce phases in execution order:

- **a.** Mapper → Reducer → Partitioner → Shuffle&Sort → Combiner
- **b.** Mapper → Partitioner → Combiner → Reducer → Shuffle&Sort
- **c.** Mapper → Shuffle&Sort → Reducer → Combiner → Partitioner
- **d.** Mapper → Combiner → Partitioner → Shuffle&Sort → Reducer

Answer: (d)

Explanation. Execution order: (1) **Mapper** reads input and emits intermediate (k, v). (2) **Combiner** (optional) locally pre-aggregates mapper output to reduce network traffic. (3) **Partitioner** assigns each key to a specific reducer. (4) **Shuffle & Sort** moves and sorts the intermediate data by key so each reducer gets all values for its keys. (5) **Reducer** processes each key group and writes the final output.

Section III — Triggers & MapReduce

Schema

```
Highschooler(ID, name, grade)  -- one row per student
Friend(ID1, ID2)              -- mutual friendship
                               -- (A,B) is also stored as (B,A)
Likes(ID1, ID2)               -- ID1 likes ID2 (one-directional)
```

Q1 [5 pts] — Trigger: delete a student when grade > 12

We need an AFTER UPDATE trigger on the grade column. Whenever a row's grade is set above 12, the student has graduated, so the row is deleted from Highschooler.

```
CREATE TRIGGER delete_graduated
AFTER UPDATE OF grade ON Highschooler
FOR EACH ROW
WHEN (NEW.grade > 12)
BEGIN
    DELETE FROM Highschooler
    WHERE ID = :NEW.ID;
END;
```

Why this works

- **AFTER UPDATE OF grade** — the trigger fires only when the grade column changes, not on every column update.
- **FOR EACH ROW** — row-level granularity, so we have access to NEW/OLD pseudo-records for the row that just changed.
- **WHEN (NEW.grade > 12)** — the trigger body runs only for rows whose new grade is past senior year. Equivalently you can wrap the DELETE in IF NEW.grade > 12 THEN ... END IF;.
- If Friend and Likes have foreign keys to Highschooler with ON DELETE CASCADE, related rows will be removed automatically; otherwise the schema designer must add additional triggers or FK actions.

Q2 [5 pts] — MapReduce: number of friends per student

Because the Friend table stores friendships in both directions — both (A,B) and (B,A) are present — we can count friends with the simplest possible MapReduce job. Every row contributes +1 to the friend count of ID1.

```
map(key, value):
    # value is one row from the Friend table: (ID1, ID2)
    (ID1, ID2) = parse(value)
    emit(ID1, 1)

reduce(key, values):
    # key    = a student ID
    # values = a list of 1's, one per friend relationship
    count = 0
    for v in values:
        count = count + v
    emit(key, count)      # output: (student_ID, #friends)
```

Why this works

Because both (A,B) and (B,A) are stored, the mapper that processes (A,B) emits (A, 1) and the mapper that processes (B,A) emits (B, 1). After shuffle, each student's reducer receives one '1' per friend relationship. Summing the list gives the friend count, with no special handling for symmetry.

(If the table were stored only one way, the mapper would have had to emit (ID1, 1) and (ID2, 1) for each row.)

Q3 [5 pts] — MapReduce: find mutual likes

Goal: output every pair (A, B) such that both (A likes B) and (B likes A) appear in the Likes table — once per mutual pair, in lexicographic order.

Idea. If we make the *unordered* pair (min(ID1,ID2), max(ID1,ID2)) the intermediate key, both directions of the same friendship hash to the *same* reducer. The reducer then receives the original directed pairs as values and only has to check that both directions are present.

```
map(key, value):
    # value is one row from the Likes table: (ID1, ID2)
    (ID1, ID2) = parse(value)
    A = min(ID1, ID2)
    B = max(ID1, ID2)
    emit( (A, B), (ID1, ID2) )      # key = unordered pair

reduce(key, values):
    # key = (A, B) with A < B
    # values = list of directed (id1, id2) pairs that map to this key
    forward = False                 # did A like B?
    backward = False                 # did B like A?
    (A, B) = key
    for (id1, id2) in values:
        if id1 == A and id2 == B: forward = True
        if id1 == B and id2 == A: backward = True
    if forward and backward:
        emit(key, "mutual")         # output: (A, B)
```

Why this works

- Both (A,B) and (B,A) generate the same key (min, max), so MapReduce delivers them to the *same* reducer.
- The reducer flags whether each direction was seen and emits the pair exactly once when both flags are true.
- Using (min, max) as the key automatically produces the pair in lexicographic order, so we never output a duplicate (B, A).

Section IV — Indexes and B+ Trees

Starting tree (degree $d = 2$)

Each non-root internal node holds 2 to 4 keys (3 to 5 child pointers); each leaf holds 2 to 4 keys.

```
Level 0 (root):      [ 20 ]
Level 1 (internal):  [ 6, 10 ]                [ 30, 40 ]
Level 2 (leaves):
    [ 2, 4 ] -> [ 6, 8, 9 ] -> [ 10, 13 ] -> [ 20, 23 ]
                -> [ 31, 32 ] -> [ 43, 54, 69, 87 ]
```

Q1 [3 pts] — Range query: $10 \leq \text{key} < 40$

B+ tree range queries have two phases: (i) descend from root to the leftmost leaf containing a key $\geq lo$, then (ii) walk the leaf-level sibling pointers until a key reaches the upper bound.

Step-by-step traversal

- Visit **root [20]**. $10 < 20$, so follow the *left* child pointer.
- Visit **internal [6, 10]**. $10 \geq 10$ (and $10 \geq 6$), so follow the *rightmost* child pointer of this node, which leads to leaf [10, 13].
- Read **leaf [10, 13]** — keys 10 and 13 are both in the range. Output them.
- Follow the next-leaf pointer to **leaf [20, 23]** — both keys still < 40 . Output them.
- Follow the next-leaf pointer to **leaf [31, 32]** — both keys still < 40 . Output them.
- Follow the next-leaf pointer to **leaf [43, 54, 69, 87]**. The first key 43 is ≥ 40 , so **STOP**.

Result. Keys returned: 10, 13, 20, 23, 31, 32. Nodes visited: root + internal [6,10] + four leaves ([10,13], [20,23], [31,32]) — i.e. **5 nodes traversed** (plus the leaf [43,54,69,87] is read only to confirm we have passed the upper bound, depending on convention).

This is exactly why B+ trees are the standard structure for indexes that must support range queries: descent cost is $O(\log_d n)$, and the rest is a sequential scan.

Q2(i) [6 pts] — Insert key 7, then key 70

Inserting 7

- Locate the target leaf: root [20] $\rightarrow$ left ($7 < 20$) $\rightarrow$ internal [6, 10] $\rightarrow$ middle child ($6 \leq 7 < 10$) $\rightarrow$ **leaf [6, 8, 9]**.
- That leaf has 3 keys; the maximum is $2d = 4$, so we have room for one more — **no split needed**.
- Insert 7 in sorted order: leaf becomes **[6, 7, 8, 9]**.

After inserting 7:

```
Root:      [ 20 ]
Internal:  [ 6, 10 ]                [ 30, 40 ]
Leaves:
    [2,4] -> [6,7,8,9] -> [10,13] -> [20,23] -> [31,32] -> [43,54,69,87]
```

Inserting 70

- Locate the target leaf: root [20] $\rightarrow$ right ($70 > 20$) $\rightarrow$ internal [30, 40] $\rightarrow$ rightmost child ($70 > 40$) $\rightarrow$ **leaf [43, 54, 69, 87]**.
- That leaf is already full (4 keys = $2d$). Inserting 70 tentatively gives **[43, 54, 69, 70, 87]** — 5 keys, an overflow. **Split**.

- Split point: take the smallest key of the right half = 69. Left half keeps [43, 54]; right half = [69, 70, 87]. The separator **69** is *copied up* to the parent (it is a leaf split, so the key remains in the right leaf as well).
- Parent internal node was [30, 40]; it becomes **[30, 40, 69]**, which is within the 4-key max — no further splits.
- The root is unchanged.

After also inserting 70:

```
Root:      [ 20 ]
Internal:  [ 6, 10 ]           [ 30, 40, 69 ]
Leaves:
  [2,4] -> [6,7,8,9] -> [10,13] -> [20,23] -> [31,32]
          -> [43,54] -> [69,70,87]
```

Q2(ii) [6 pts] — From the original tree: delete 13, then delete 8

Deleting 13

- Key 13 lives in **leaf [10, 13]**. After deletion the leaf has only 1 key — **underflow** (minimum is $d = 2$).
- Look at the left sibling **[6, 8, 9]**: it has 3 keys, more than the minimum, so we can **redistribute** (borrow) instead of merging.
- Borrow the largest key from the left sibling — **9**. Left sibling becomes [6, 8]; the underflowing leaf becomes [9, 10].
- Update the parent separator that used to be 10 (the old smallest key of the right leaf): the new smallest key of the right leaf is 9, so the separator changes from 10 to 9. The internal node [6, 10] becomes **[6, 9]**.

After deleting 13:

```
Root:      [ 20 ]
Internal:  [ 6, 9 ]           [ 30, 40 ]
Leaves:
  [2,4] -> [6,8] -> [9,10] -> [20,23] -> [31,32] -> [43,54,69,87]
```

Deleting 8 (continuing from the original tree)

The exam asks for each deletion independently from the original tree, so we restart from the starting tree and remove 8.

- Key 8 lives in **leaf [6, 8, 9]**. After deletion the leaf has 2 keys = d , the minimum — **no underflow**.
- No redistribution, no merge, no parent updates — the leaf simply becomes [6, 9].

After deleting 8 (from the ORIGINAL tree):

```
Root:      [ 20 ]
Internal:  [ 6, 10 ]          [ 30, 40 ]
Leaves:
  [2,4] -> [6,9] -> [10,13] -> [20,23] -> [31,32] -> [43,54,69,87]
```

Why these rules

- **Insert / split (leaf)**: copy-up the smallest key of the right half to the parent — leaves keep *all* the data.
- **Insert / split (internal)**: push-up the middle key — internal nodes only route, and a routing key exists in exactly one place.
- **Delete / redistribute**: borrow from a sibling with $> d$ keys; update the parent separator to the new smallest key of the right neighbour.

- **Delete / merge:** if neither sibling can lend, merge with one and pull the parent separator down. Recurse upward if the parent now underflows.
- **Tree height** only changes by splitting the root (grows) or by merging children of a 1-key root (shrinks).

Appendix — Quick Reference Card

Concept	Key points to remember
Trigger	Auto-fires on INSERT/UPDATE/DELETE; BEFORE vs AFTER; FOR EACH ROW vs FOR EACH STATEMENT; OLD/NEW pseudo-records.
Stored procedure	Named, pre-compiled SQL block; called explicitly; CREATE PROCEDURE is a separate privilege; centralises business logic.
Scalar UDF	Returns exactly ONE scalar value; usable inside any SQL expression. For multiple return values use a table-valued function or a procedure with OUT params.
ACID	Atomicity (all or nothing), Consistency (invariants preserved), Isolation (no concurrent interference), Durability (committed changes survive crashes).
Auto-commit	Without explicit BEGIN TRANSACTION, every SQL statement is its own transaction.
Isolation levels	READ UNCOMMITTED < READ COMMITTED < REPEATABLE READ < SERIALIZABLE; higher = fewer anomalies, lower throughput.
Anomalies	Dirty read, lost update, non-repeatable read, phantom read — each is removed by progressively stricter levels.
Index purpose	Speeds up SELECT (especially WHERE/JOIN); slows down INSERT/UPDATE/DELETE because the index must be maintained.
Unique constraint	DBMS automatically backs it with a (usually non-clustered) unique index for fast duplicate detection.
B+ tree	All data in leaves; leaves linked; perfectly height-balanced; supports both equality and range queries in $O(\log_d n)$ I/Os.
B+ tree insert	Find leaf → insert → split if overflow → leaves COPY-UP, internals PUSH-UP. Tree only grows by root split.
B+ tree delete	Find leaf → delete → if underflow, redistribute from sibling, else merge and recurse upward. Tree only shrinks by root merge.
MapReduce	User writes map() and reduce(); framework handles parallelism, shuffle, and fault tolerance. Mapper may emit 0/1/many pairs.
MR phases	Map → Combiner (optional) → Partitioner → Shuffle & Sort → Reduce.
Combiner	Local mini-reducer that pre-aggregates mapper output; must be associative and commutative.
Partitioner	Routes intermediate keys to specific reducers; default = hash(key) mod R.

Data locality	Hadoop schedules mappers on (or near) the nodes that hold the input HDFS blocks, minimising network I/O.
----------------------	--